E.E.S.T. N.º 1 “Eduardo Ader” · Vicente López
Laboratorio de Programación (LPR) · 5.º año, 3.º, grupo A

MANUAL DEL PROGRAMADOR

Manual técnico del motor del juego escrito en C++

Documento técnico · Versión 1.0.0 · 21 de octubre de 2026

Proyecto	Juego de Naves en Consola (Actividad 8)
Materia	Laboratorio de Programación (LPR)
Curso y grupo	5.º año, 3.º, grupo A
Institución	E.E.S.T. N.º 1 “Eduardo Ader” — Vicente López
Estudiante	Cheheid, Malek
Docente	Mansilla Muñoz York Elías
Versión	1.0.0
Fecha	21 de octubre de 2026

Repositorios del proyecto

https://github.com/malekcheheid-art/juegodenaves.git

https://gitlab.com/malekcheheid-art/juegodenaves.git

Licencia MIT

Contenido

1. Introducción 4

1.1 Propósito del documento 4

1.2 Alcance 4

1.3 Requisitos de lectura 4

2. Requisitos del entorno 4

3. Compilación y ejecución 4

3.1 Compilar 4

3.2 Ejecutar 5

4. Estructura del repositorio 5

5. Arquitectura del software 6

5.1 Vista general 6

5.2 Clase Nave (el jugador) 6

5.3 Clase Asteroide 7

5.4 Clase Bala 7

5.5 Funciones libres 8

6. Flujo de ejecución 8

6.1 Inicialización 8

6.2 Bucle principal 8

6.3 Fin de la partida 8

7. Decisiones de diseño 9

7.1 Encapsulamiento 9

7.2 Constructores y nombres de parámetros 9

7.3 Pasaje por referencia 9

7.4 Memoria dinámica 9

7.5 Entrada de teclado sin bloqueo 9

7.6 Detección de colisiones 9

8. Funcionalidad única implementada: marcador de puntos 10

8.1 Descripción 10

8.2 Puntos de inserción en el código 10

8.3 Prueba correspondiente 10

9. Guía de extensión para otro programador 10

10. Pruebas y evidencias 11

11. Control de versiones y publicación 11

12. Limitaciones conocidas 12

Referencias 12

1. Introducción

1.1 Propósito del documento

Este manual describe el diseño, la estructura y las decisiones de implementación del proyecto **Juego de Naves en Consola**, desarrollado como obligación de la Actividad 8 de la materia Laboratorio de Programación (LPR). Su destinatario es toda persona que deba leer, mantener o ampliar el código fuente: docentes evaluadores, compañeros de curso o futuros desarrolladores del proyecto.

1.2 Alcance

El documento cubre el archivo `src/main.cpp` (372 líneas), que contiene la totalidad del programa: las clases **Nave**, **Asteroide** y **Bala**, las funciones auxiliares de consola y la función `main()` con el bucle principal. No reproduce el código completo —está disponible en el repositorio y en el informe APA de la actividad—, sino los fragmentos necesarios para explicar cada concepto y cada punto de extensión.

1.3 Requisitos de lectura

Se supone conocimiento de programación imperativa en C++ (tipos, ciclos, funciones, arreglos) y una noción introductoria de programación orientada a objetos: clases, objetos, atributos y métodos.

2. Requisitos del entorno

El programa está escrito para consola de Windows y no requiere librerías de terceros.

Componente	Requerido	Detalle
Sistema operativo	Microsoft Windows	Se usa la API de consola (`windows.h`) y `conio.h`, propias de Windows.
Compilador	g++ con soporte de C++	Compilado y probado con TDM-GCC 10.3. También puede compilarse con MSVC o Code::Blocks.
Biblioteca estándar	`iostream`, `vector`, `string`, `ctime`, `cctype`	Incluidas en la propia distribución del compilador.
Dependencias externas	Ninguna	No se emplean librerías de terceros, motores de juego ni recursos gráficos.
Resolución de consola	Recomendada 80 x 25	El área de juego ocupa hasta la columna 71 y la fila 23.

3. Compilación y ejecución

3.1 Compilar

Desde la carpeta raíz del repositorio, con MinGW:

```
g++ src/main.cpp -o juego_naves.exe
```

En Visual Studio basta con abrir `src/main.cpp` y compilar en modo *Consola* (/SUBSYSTEM:CONSOLE).

3.2 Ejecutar

Desde la misma carpeta, en la consola:

```
juego_naves.exe
```

Al arrancar, el programa solicita el nombre y apellido del piloto, limpia la consola, oculta el cursor y comienza la partida. El nombre solicitado queda visible en el HUD superior y es el que debe aparecer en la captura de evidencia exigida por la consigna.

> **Nota:** el juego está pensado para una consola de tamaño estándar. Si la ventana es muy pequeña, parte del área de juego queda fuera de pantalla.

4. Estructura del repositorio

El orden de carpetas respeta la estructura obligatoria de la consigna: no se aceptan archivos sueltos.

```
juegodenaves/
|-- .gitattributes      <- binarios (pdf, png) protegidos del CRLF
|-- .gitignore          <- exclusiones (.exe, *.out, .vscode/)
|-- LICENSE             <- licencia MIT
|-- README.md           <- portada del proyecto
|-- docs/
|   |-- InformeEEST1_LPR2026_ACT08_CheheidMalek_v1.0.0.pdf
|   |-- CHANGELOG.md    <- historial de versiones
|   `-- manuales/
|       |-- manual_programador_v1.0.0.pdf
|       `-- manual_usuario_v1.0.0.pdf
|-- src/
|   `-- main.cpp        <- código fuente completo
`-- capturas/
    `-- combate_consola.png
```

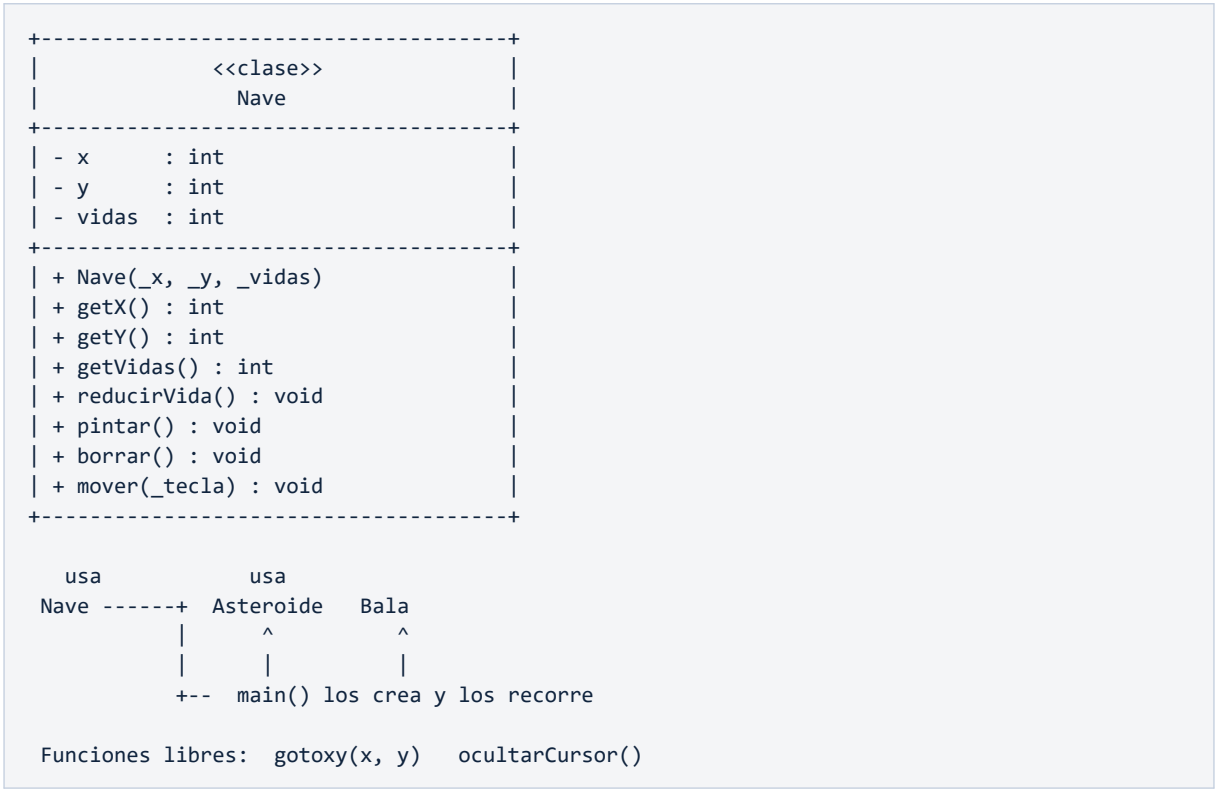
El proyecto se publica de forma simultánea en dos remotos individuales:

- **GitHub:** https://github.com/malekcheheid-art/juegodenaves.git
- **GitLab:** https://gitlab.com/malekcheheid-art/juegodenaves.git

5. Arquitectura del software

5.1 Vista general

El programa está formado por tres clases y dos funciones libres. Las clases se concentran en un único archivo de traducción para mantener la entrega en un solo archivo, tal como pide la actividad.



5.2 Clase Nave (el jugador)

Es la clase con más comportamiento. Sus atributos son **privados**: desde `main()` no se puede escribir `jugador.x = 99;`, sólo se llega a ellos a través de los métodos públicos. Ese es el encapsulamiento.

Miembro	Visibilidad	Firma	Responsabilidad
x, y	private	int	Coordenadas de la esquina superior izquierda de la nave.
vidas	private	int	Vidas restantes del jugador.
Constructor	public	Nave(int _x, int _y, int _vidas)	Inicializa los tres atributos. Sin él, quedarían con valores indeterminados.
getX(), getY(), getVidas()	public	int	Accesores de sólo lectura; devuelven el valor sin permitir modificarlo.
reducirVida()	public	void	Descuenta una vida (vidas--).

Miembro	Visibilidad	Firma	Responsabilidad
pintar()	public	void	Dibuja la nave en tres filas: punta, cuerpo y base.
borrar()	public	void	Escribe espacios en las mismas posiciones; evita estelas.
mover(char tecla)	public	void	Valida los límites del área de juego y actualiza la posición.

```
class Nave {
private:
    int x, y;
    int vidas;
public:
    Nave(int _x, int _y, int _vidas) {
        x = _x;  y = _y;  vidas = _vidas;
    }
    int getX() { return x; }
    int getVidas() { return vidas; }
    void reducirVida() { vidas--; }
    void mover(char tecla) {
        borrar();
        if (tecla == 'a' && x > 2)  x--;
        if (tecla == 'd' && x < 70) x++;
        if (tecla == 'w' && y > 3)  y--;
        if (tecla == 's' && y < 20) y++;
        pintar();
    }
};
```

5.3 Clase Asteroide

Enemigo que desciende una fila por vuelta del bucle. Al pasar de la fila 23 se reinicia arriba, en una columna aleatoria entre 2 y 71, de modo que la partida nunca se agota.

Miembro	Visibilidad	Responsabilidad
x, y	private	Posición actual del asteroide.
Asteroide(int _x, int _y)	public	Inicializa la posición.
getX(), getY()	public	Accesores de lectura.
pintar(), borrar()	public	Dibujan o borran el carácter `0`.
mover()	public	Incrementa `y`; si supera la fila 23, se recoloca arriba con `rand() % 70 + 2`.

5.4 Clase Bala

Proyectil que asciende una fila por vuelta y se descarta apenas sale por arriba del área visible.

Miembro	Visibilidad	Responsabilidad
x, y	private	Posición actual del proyectil.
Bala(int _x, int _y)	public	Nace sobre la nave, dos columnas a la derecha y una fila arriba de su punta.
mover()	public	Borra la posición anterior, decrementa y y dibuja |.
fueraDeRango()	public	Devuelve `true` cuando `y < 4`; es la señal para liberar la memoria.

5.5 Funciones libres

- `gotoxy(int x, int y)`: ubica el cursor en la consola mediante `SetConsoleCursorPosition`. Se declara antes de las clases porque sus métodos la invocan.
- `ocultarCursor()` configura `CONSOLE_CURSOR_INFO` con el operador `&`, que es un ejemplo directo de pasaje por referencia.

6. Flujo de ejecución

6.1 Inicialización

1. Se siembra el generador aleatorio con `srand(time(NULL))`.
2. Se solicita el nombre del piloto con `getline(cin, ...)`, que admite espacios.
3. Se limpia la consola y se oculta el cursor.
4. Se crea el objeto jugador con el constructor: `Nave jugador(35, 20, 3);`
5. Se cargan 5 asteroides en posiciones aleatorias con `new`.
6. Se dibuja la nave y se entra al bucle principal.

6.2 Bucle principal

Mientras queden vidas, cada vuelta ejecuta cinco tareas en este orden preciso:

```
while (!gameOver) {
    1. mostrar HUD (piloto, vidas, score)
    2. leer teclado       -> mover o disparar
    3. mover las balas    -> borrar las que salieron
    4. mover los asteroides y detectar colisiones
    5. Sleep(30)          -> pausa para no saturar la CPU
}
```

El orden importa: las balas se mueven antes de comprobar las colisiones, de modo que un impacto se resuelve en la misma vuelta en que se produce.

6.3 Fin de la partida

Cuando `vidas` llega a cero se marca `gameOver = true`, se sale del bucle, se limpia la pantalla y se muestra el mensaje de fin junto a la pausa de `system("pause")`.

7. Decisiones de diseño

7.1 Encapsulamiento

Los tres atributos de **Nave** son privados y el único camino para leerlos son los accesores. Así se garantiza que ninguna parte del programa pueda alterar el estado del jugador de forma indirecta; por ejemplo, impidiendo que un error de tipeo en `main()` asigne vidas negativas.

7.2 Constructores y nombres de parámetros

Las tres clases definen un constructor parametrizado. Los parámetros se llaman `_x`, `_y` y `_vidas`: el prefijo de guion bajo los distingue de los atributos homónimos dentro del cuerpo del método, evitando la ambigüedad más común de quien se inicia en POO.

7.3 Pasaje por referencia

Las APIs de Windows devuelven o esperan estructuras. En `ocultarCursor()`, el operador `&` sobre `cci` entrega la dirección de la variable, de modo que la función modifica el objeto original y no una copia.

7.4 Memoria dinámica

La cantidad de asteroides y balas cambia en tiempo de ejecución, por lo que se usan `vector<Asteroide*>` y `vector<Bala*>`. Cada elemento se crea con `new` y se libera con `delete` antes de eliminarlo del vector.

> **Cuidado:** borrar el elemento del vector sin antes hacer `delete` produce una fuga de memoria: el espacio queda reservado pero inutilizable.

7.5 Entrada de teclado sin bloqueo

`_kbhit()` informa si hay una tecla pendiente de lectura; de este modo el juego nunca se detiene esperando al usuario. La tecla se convierte con `tolower()` para que el Bloq Mayús no altere los comandos.

7.6 Detección de colisiones

La nave ocupa 4 columnas por 3 filas. El impacto se detecta comprobando que las coordenadas del asteroide caigan dentro de ese rectángulo:

```
if (ast->getX() >= jugador.getX() &&
    ast->getX() <= jugador.getX() + 4 &&
    ast->getY() >= jugador.getY() &&
    ast->getY() <= jugador.getY() + 2) {
    jugador.reducirVida();
}
```

Para bala contra asteroide la condición es de igualdad exacta de columna, más una diferencia de una fila a favor del asteroide, porque ambos cuerpos miden dos filas de alto.

8. Funcionalidad única implementada: marcador de puntos

8.1 Descripción

Se eligió la funcionalidad de **marcador de puntos (SCORE)** de las quince opciones publicadas en el anexo de la actividad. Consiste en acumular puntos por cada asteroide destruido y mostrarlos en pantalla durante toda la partida.

8.2 Puntos de inserción en el código

Línea	Elemento	Qué hace
246	`int score = 0;`	Declara el acumulador antes del bucle, junto a `gameOver`, para que se reinicie en cada partida.
272-280	HUD del bucle	Escribe `SCORE: n` en la fila 2, columna 15, usando `gotoxy()` para no pisar el resto del HUD.
353	`score += 10;`	Suma el puntaje dentro del bloque de colisión bala-asteroide, inmediatamente después de destruir el objetivo y antes del `break`.

La regla de puntuación es de **10 puntos por impacto** confirmado. La variable es local a `main()`; si en el futuro se quisiera reiniciarla o persistirlo, lo correcto sería encapsularla en una clase Partida en lugar de exponerla.

8.3 Prueba correspondiente

Al disparar sobre un asteroide el marcador pasa de 0 a 10; al segundo impacto, a 20. El valor queda visible en la captura de evidencia adjunta.

9. Guía de extensión para otro programador

Para incorporar una funcionalidad nueva sin romper lo existente:

- Identificar el punto exacto dentro del bucle (entrada, movimiento, colisión o HUD) donde la nueva lógica debe ejecutarse.
- Declarar el estado nuevo *antes* del `while`, para que se reinicie en cada partida.
- Si la lógica pertenece a un objeto, agregar el atributo como **private** y exponerlo mediante un método público; no abrir el atributo a `public`.
- Actualizar el HUD sin pisar los campos existentes: reservar una columna libre y usar `gotoxy()`.
- Compilar y probar al menos dos partidas completas antes de publicar.
- Registrar el cambio en `docs/CHANGELOG.md` y actualizar la sección de estado del `README.md`.

10. Pruebas y evidencias

Prueba	Resultado
Compilación con g++ (TDM-GCC 10.3)	Correcta, sin errores.
Arranque y petición del nombre del piloto	Se solicita y se muestra en el HUD.
Movimiento con W / A / S / D	Respeta los límites del área de juego.
Disparo con ESPACIO	La bala asciende y se libera al salir de rango.
Colisión asteroide-nave	Descuenta una vida en cada impacto.
Colisión bala-asteroide	Destruye el objetivo y suma 10 puntos al marcador.
Fin de partida con 0 vidas	Muestra el mensaje de GAME OVER.

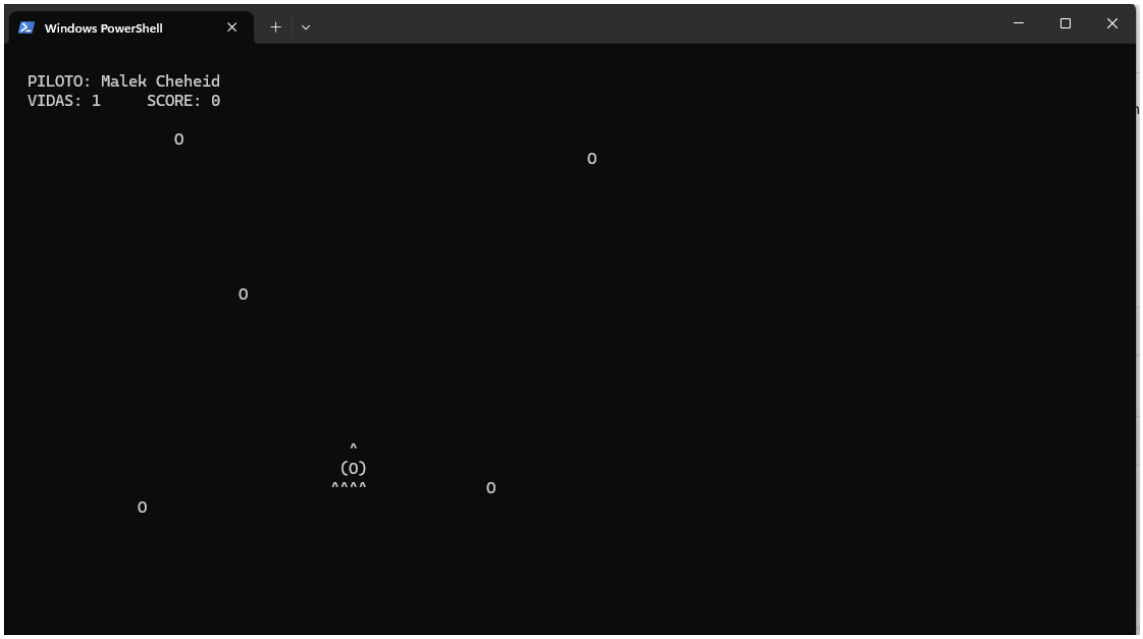


Figura 1. Ejecución de la partida en consola. Se observa el HUD con el nombre del piloto, el contador de vidas y el marcador de puntos.

11. Control de versiones y publicación

El proyecto sigue **SemVer** y se publica de forma simultánea en GitHub y GitLab. La versión actual es la **1.0.0**, etiquetada como `v1.0.0`.

Elemento	Valor
Versión	1.0.0

Elemento	Valor
Etiqueta Git	v1.0.0 (anotada)
Rama	main
Remoto 1	https://github.com/malekcheheid-art/juegodenaves.git
Remoto 2	https://gitlab.com/malekcheheid-art/juegodenaves.git
Historial	docs/CHANGELOG.md

12. Limitaciones conocidas

- **Sólo Windows.** Las funciones `gotoxy()`, `ocultarCursor()` y `Sleep()` dependen de la API de consola de Windows; llevarlo a Linux exigiría reemplazarlas por `ncurses`.
- **No libera toda la memoria al finalizar.** Las balas y asteroides vivos en el momento de salir no se liberan; el sistema operativo recupera el espacio al terminar el proceso, pero una versión de mayor duración debería recorrer ambos vectores antes de salir.
- **Sin pausa ni reinicio.** Una vez finalizada la partida hay que volver a ejecutar el programa.
- **Área de juego fija.** Los límites (columna 2 a 70, fila 3 a 23) están codificados como constantes literales dentro de los métodos.
- **Un solo archivo de traducción.** Las tres clases conviven en `main.cpp`; en un proyecto mayor convendría separarlas en encabezados y unidades de implementación.

Referencias

Recurso	Dirección
Repositorio en GitHub	https://github.com/malekcheheid-art/juegodenaves.git
Repositorio en GitLab	https://gitlab.com/malekcheheid-art/juegodenaves.git
cplusplus.com — Clases y objetos en C++	https://cplusplus.com/doc/tutorial/classes/
GeeksforGeeks — Modificadores de acceso en C++	https://www.geeksforgeeks.org/access-modifiers-in-c/
Microsoft Learn — Constructores e inicialización de clases	https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp
Documentación de Git	https://git-scm.com/doc
Keep a Changelog	https://keepachangelog.com/
Semantic Versioning	https://semver.org/lang/es/